

Crawler Extension Summary

Overview

Extended the Vysalytica crawler to handle up to 5 pages per website instead of the previous limit. The crawler now properly follows internal links and processes multiple pages per audit.

Changes Made

1. engine_crawl.py - Crawler Engine

- **Modified `discover_pages()` function:**
 - Added `max_pages` parameter (optional, configurable)
 - Default behavior now uses `CRAWL_MAX_PAGES` environment variable (default: 5)
 - Replaced all hardcoded “12” references with dynamic `max_pages` variable
 - Crawler now efficiently discovers pages from:
 - Homepage (always included)
 - Sitemap.xml (if available)
 - Internal links from homepage (following depth-first strategy)

2. plans.py - Plan Configuration

- **Updated Full Audit plan:**
 - Changed `max_pages` from 12 to 5
 - Updated description: “Comprehensive audit of up to 5 pages”
 - Updated `get_plan_recommendation()` logic to reflect new 5-page limit
 - Updated `compare_plans()` display to show “Up to 5” for Full plan
- **Plan Limits:**
 - QuickScan: 3 pages (unchanged)
 - Full Audit: 5 pages (changed from 12)
 - Agency: 12 pages (unchanged for enterprise usage)

3. .env.example - Configuration

Added comprehensive crawler configuration options:

```
CRAWL_MAX_PAGES=5           # Default max pages to discover
CRAWL_CONCURRENT=false      # Enable concurrent fetching
CRAWL_MAX_WORKERS=4         # Thread pool size for concurrent fetching
FETCH_RETRY_COUNT=3         # Retry attempts for HTTP fetches
FETCH_BACKOFF_SECONDS=0.5   # Backoff delay for retries
ROBOTS_SITEMAP_CACHE_TTL_SECONDS=86400 # Cache TTL for robots.txt/sitemap.xml
```

4. Test Scripts

Created two comprehensive test scripts:

- **test_crawler_5pages.py:** Unit tests for crawler limits

- Tests default behavior (5 pages)
- Tests explicit max_pages parameter
- Tests environment variable override
- Verifies limit enforcement
- **test_crawler_detailed.py**: Integration tests with real websites
- Tests crawler with multiple websites
- Verifies internal link discovery
- Confirms max_pages limit is respected
- Validates crawler behavior with sites of varying complexity

How It Works

Page Discovery Strategy

1. **Start with homepage** - Always included as the first page
2. **Check sitemap.xml** - Discover pages from sitemap if available (up to max_pages - 1)
3. **Follow internal links** - Crawl homepage for internal links, organized by depth:
 - Depth 0: Root-level pages
 - Depth 1: One level deep
 - Depth 2: Two levels deep
 - Depth 3+: Three or more levels deep
4. **Deduplicate** - Normalize URLs to prevent duplicates (handles trailing slashes, fragments, etc.)
5. **Limit enforcement** - Stop discovery when max_pages is reached

Configuration Hierarchy

The crawler determines the max_pages limit in this order:

1. Explicit max_pages parameter passed to discover_pages()
2. CRAWL_MAX_PAGES environment variable
3. Default value: 5

Testing Results

All tests passed successfully! ✓

Test Results:

- **Default behavior**: Correctly limits to 5 pages
- **Explicit limits**: Respects custom max_pages values (3, 5, 10)
- **Environment override**: Properly uses CRAWL_MAX_PAGES env var
- **Real website test**: Successfully discovered 5 pages from python.org following internal links
- **Simple sites**: Handles sites with limited internal links (e.g., example.com)

Usage Examples

In Code:

```
from api import engine_crawl

# Use default (5 pages)
pages = engine_crawl.discover_pages("https://example.com")

# Explicit limit
pages = engine_crawl.discover_pages("https://example.com", max_pages=5)

# With environment variable set
os.environ["CRAWL_MAX_PAGES"] = "7"
pages = engine_crawl.discover_pages("https://example.com") # Will discover up to 7
pages
```

With Plans:

```
from vysalytica import plans

# QuickScan: 3 pages
plan_limits = plans.get_plan_limits("quickscan")
# Returns: {"max_pages": 3, ...}

# Full Audit: 5 pages
plan_limits = plans.get_plan_limits("full")
# Returns: {"max_pages": 5, ...}

# Agency: 12 pages
plan_limits = plans.get_plan_limits("agency")
# Returns: {"max_pages": 12, ...}
```

Benefits

1. **Configurable:** Easy to adjust via environment variables or code
2. **Efficient:** Discovers only necessary pages, reducing crawl time
3. **Smart:** Follows internal links with depth-based prioritization
4. **Robust:** Handles sites with or without sitemaps
5. **Deduplication:** Normalizes URLs to prevent duplicate page processing
6. **Backward Compatible:** Existing code continues to work with new defaults

Git Commit

Changes committed to local repository:

- Commit: a6aacd5
- Branch: main
- Message: "Extend crawler to handle up to 5 pages per website"

Next Steps (Optional)

1. Push changes to remote repository

2. Update documentation/README if needed
3. Deploy to staging environment for testing
4. Monitor crawler performance in production
5. Consider adding metrics/logging for page discovery statistics